

滴滴实习项目简历备战指南

适用场景：秋招面试准备 · 技术面 · 项目复盘
核心目标：从简历项目到面试问答的完整链路
文档版本：2026-09-15

目录

- [1. 项目背景与定位](#)
- [2. 项目一：性能监控与体验优化](#)
- [3. 项目二：AI 智能监控降噪](#)
- [4. 技术亮点提炼](#)
- [5. 常见面试问题清单](#)
- [6. 避坑指南](#)
- [7. 快速复习 Checklist](#)

项目背景与定位

实习时间线

滴滴出行 · 企业版商旅 | 前端开发实习生 | 2026.01 — 2026.07

业务上下文

- 业务线**：商旅（机票/酒店/火车票 H5 + 小程序）
- 用户规模**：企业用户预订场景
- 技术栈**：MPX（类 Vue 小程序框架）+ H5 双端
- 团队定位**：C 端业务前端，关注预订链路体验与稳定性

两个项目的关系

1	
2	项目一：性能监控与体验优化
3	定位：搭建函数级性能基础设施 + 具体页面优化
4	产出：LCP↓17%、卡顿↓27%
5	
6	↓ 沉淀方法论
7	
8	项目二：AI 智能监控降噪
9	定位：LLM 驱动的告警判定系统
10	产出：降噪引擎 + 火车票 Domain 配置
11	

共性：都是「从问题定位到工程化沉淀」的完整闭环，体现前端基础设施建设能力。

项目一：性能监控与体验优化

1.1 一句话概括

在商旅 H5/小程序搭建函数级性能监控体系，定位并优化列表页 LCP 和星河底部菜单卡顿，沉淀为可复用 Agent Skill。

1.2 问题背景（面试必讲）

- **痛点**：列表页 LCP 约 1400ms，星河底部菜单切换卡顿，用户反馈体验差
- **缺少抓手**：此前只有 FCP/LCP 数字，缺少函数级、接口级的性能数据
- **排查困难**：不知道慢在网络、渲染还是计算；优化没有优先级

1.3 技术方案架构

核心基础设施：PerfMonitor

```
1  src/utils/perf-monitor.js
2      └─ mark(label)      // 打点
3      └─ measure(from, to) // 计算耗时
4      └─ report()         // 上报 Omega
5      |
6      └─ 跨页链路 (home→list)
7      └─ 同页体验 (LCP/FCP)
8      └─ 接口维度 (getHomeData 等)
```

双端适配（技术亮点）：

- H5 环境： `PerformanceObserver` API
- 星河小程序： `wx.getPerformance` API
- 通过 `Adapter` 模式抽象统一上报，避免写两套监控代码

性能优化三板斧

```
1  定位慢 → 找瓶颈 → 改造 → 验收
2      ↓      ↓      ↓      ↓
3  埋点    分析报告  改代码  看数字
```

1.4 具体优化案例（面试重点）

案例 A：列表页 LCP 优化

问题定位：

1. 接入 `getHomeData` 接口耗时埋点
2. 发现 `privacyPopList` 等非关键接口串在首屏关键路径
3. 切日期场景会重复请求，缓存命中时仍触发网络请求

优化动作：

- **剥离非关键路径**：把隐私弹窗接口移出 LCP 统计链路，`defer` 延后加载
- **缓存命中直渲**：增加状态机 `cacheHit → skipFetch → directRender`，避免白等

- 修正口径：切日期重渲时，区分「接口返回」和「缓存命中」两条路径分别上报

结果：LCP 1400ms → 1160ms (↓17%)

技术细节（面试追问）：

- Q: 为什么要剥离 privacyPopList?
- A: LCP 应对应用户感知的最大内容渲染，隐私弹窗不是首屏关键内容，串进来会污染口径，让优化方向跑偏。

案例 B：星河底部菜单卡顿

问题定位：

- 用户切换底部 tab 时有明显卡顿
- 通过 PerfMonitor 定位到排序计算开销大

优化动作：

- 用 useMemo 缓存已排序结果，避免每次切换重复计算

结果：卡顿改善约 27%

技术细节（面试追问）：

- Q: 为什么排序会导致卡顿?
- A: 底部菜单数据量不大，但每次切换都触发排序 + 重渲染，累积起来拖主线程；缓存后只在数据变化时才重算。

1.5 工程化沉淀

perf-monitor.js 设计要点

能力	实现
三段式 API	mark / measure / report
双端兼容	Adapter 模式抽象 H5/星河差异
跨页链路	home_to_list_complete 全链路
同页体验	LCP/FCP 自建监控
接口维度	关键接口耗时 (getHomeData 等)
上报通道	统一经 Omega 平台

Agent Skill 固化

把「定位 → 诊断 → 优化 → 验收」闭环固化为 page-performance-analysis Skill，后续性能问题可复用分析流程。

Skill 核心能力：

- 读页面源码，按加载/交互/监听三阶段拆函数
- 估算耗时并排序（第一梯队 >500ms / 第二梯队 200-500ms / 第三梯队 <200ms）
- 识别瓶颈（串行瀑布、频繁触发、防抖节流、重复请求）

- 输出 P0/P1/P2 分级优化建议

1.6 面试高频问题

Q1: 这个 Skill 是你写的吗？

诚实回答：

- Skill 框架是团队搭的，但我参与了监控体系搭建和性能优化的实际落地
- 具体问题是定位的：接口耗时、缓存逻辑、星河端兼容
- 亮点在于 H5/星河双端环境抽象、统一上报链路

Q2: 项目难点是什么？

三个难点：

1. **双端技术栈差异**：星河和 H5 API 不同，需要 Adapter 抽象统一上报逻辑
2. **场景复杂性**：列表页「切日期」场景，原来每次都发请求导致白屏，通过 PerfMonitor 数据发现后改成缓存命中直渲
3. **隐蔽性能瓶颈**：隐私弹窗接口 `privacyPopList` 拖了 LCP，但这个接口是后加的，没人注意到

Q3: 如何验证优化效果？

验证链路：

1 | 本地 DevTools 验证 → 灰度小流量 → Omega 看板对比 → 全量上线

- LCP 1400→1160ms 是线上真实数据，经 Omega 平台统计
- 卡顿改善约 27% 也是基于用户行为埋点的对比

Q4: 为什么要做 Skill？和 Lighthouse 有什么区别？

维度	Lighthouse	page-performance-analysis Skill
数据来源	真实运行测量	源码静态分析 + 类型估算
输出	指标与资源级建议	函数级耗时排序 + 代码侧改法
使用门槛	要起环境、会看面板	对话一句即可开跑
复用	偏个人操作	规程固化进 Agent Skill

价值：把性能复盘从个人经验产品化为团队可触发流程。

项目二：AI 智能监控降噪

2.1 一句话概括

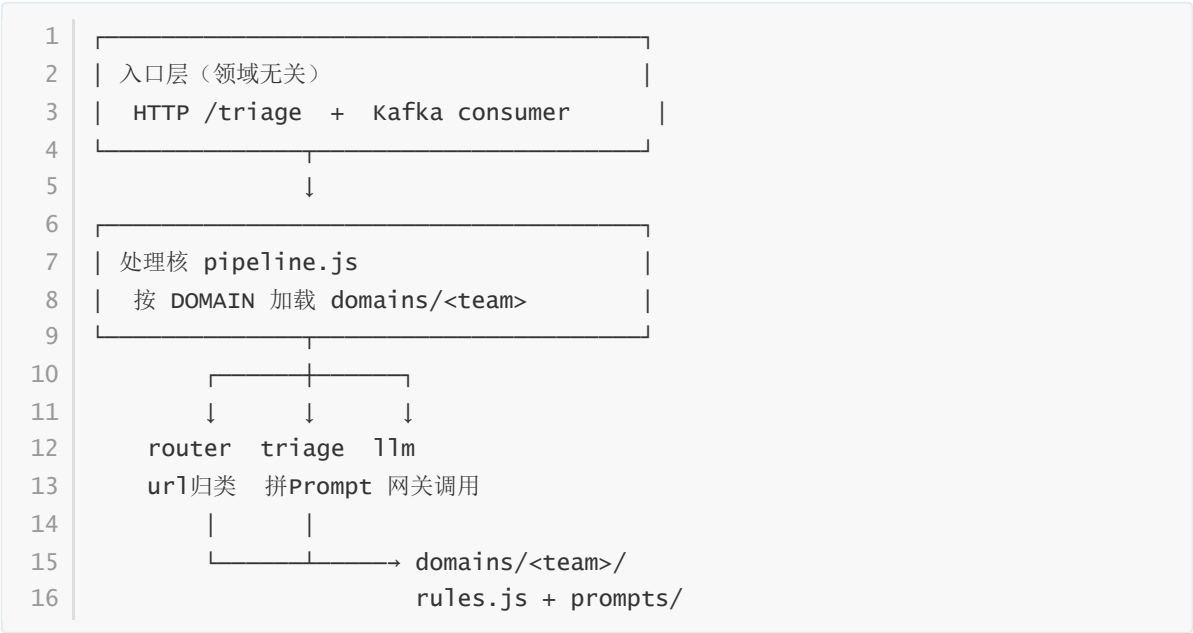
基于 LLM 的接口告警智能判定系统，每条告警走一次模型判断「该不该报、定什么级」，把噪音挡在派发之前。

2.2 问题背景（面试必讲）

- **痛点**：机酒火线上接口告警量大，噪音占比高
 - 业务正常拒绝（无票、已退改、超时未支付）占大多数
 - 低价值展示类错误（NPS、公告、分享页）刷屏
 - 真故障被淹没，值班容易漏看
- **核心矛盾**：不能用简单规则过滤（会误杀真故障），需要理解业务语义

2.3 技术方案架构

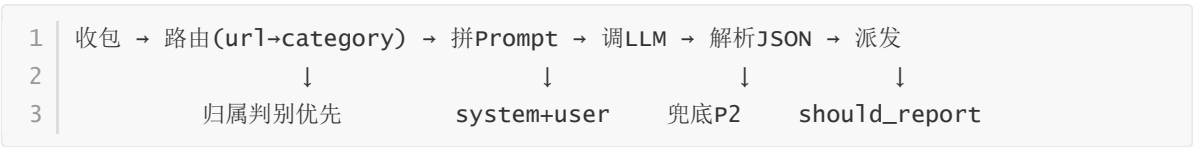
系统分层



核心设计原则：

- **引擎领域无关**：换团队 = 换 DOMAIN + 加文件夹，引擎零改动
- **判定交给模型**：工程只做路由与编排，阈值/规则全在 Prompt
- **宁可误报不可误杀**：漏报（FN）优先压到接近 0

单条告警处理流程



2.4 我的具体工作（面试重点）

工作 A：接口发现（discover Skill）

目标：从业务代码仓库自动摘取核心接口清单，避免手工枚举遗漏。

方法：

1. **探技术栈**：识别项目是前端/后端/原生 App，url 落法不同
2. **广召回**：grep 广撒网 + Explore agent 补读内联/动态拼接

3. 正交打标：

- 归属维度（是否 non-core）：纯前缀判定
- 核心 vs 边缘（severity 基线）：反查「挂了用户能否走完主流程」

4. 带证据：捕获「码 → 原文 (文件:行)」，便于后续人工核验

产出：train-urls.js / flight-urls.js / hotel-urls.js

工作 B：火车票 Domain 配置

目标：根据接口清单，配置路由规则、severity 基线、业务拒绝清单。

配置结构：

```
1 domains/train/
2   └─ rules.js           # url → category 路由表
3   └─ prompts/
4       └─ _base.md       # 通用核：schema、枚举、全局规则
5       └─ non-core.md    # 非自有接口（基线 P3）
6       └─ query.md       # 余票查询分类
7       └─ payment.md     # 支付分类
8       └─ order.md       # 下单分类
9       └─ refund-change.md # 退改分类
```

关键决策：

- ownedPrefixes = ['/esapp/']：自有前缀
- fallbackkey = 'order'：未命中规则时保守上报
- 余票查询异常判 P1（因为是下单主流程入口）

工作 C：business_reject 清单（技术亮点）

定义：接口返回了错误形态，但属于业务预期路径（无票、已退改、重复提交），不应打扰值班。

落地方式：

- 在各分类 Prompt 的 business_reject 字段列举码值与语义
- discover Skill 从业务代码捕获「码 → 原文 (文件:行)」证据

上线前必做：回源码逐条核对，避免误杀真故障。

工作 D：兜底策略

场景：模型超时、网关报错、JSON 解析失败。

处理：统一 should_report=true, severity=P2，永不吞告警、不卡 Kafka 消费。

意义：降噪能力降级时，可用性优先于降噪率。

工作 E：评测闭环

目标：用漏报率 FN 作为头号指标，防止降噪过度。

流程：

```
1 拉异常 → 指纹去重 → 标gold → run/eval → score指标 → 归因改Prompt
```

指标体系：

指标	优先级
漏报率 FN	头号，先压到 ≈ 0
误报率 FP	FN 达标后再降
severity 完全一致/差一级/差多级	主精度

2.5 架构理解（面试加分项）

为什么每条都走 LLM？

决策：不做规则预过滤，每条都走 LLM。

理由：

- 1. 业务拒绝语义复杂（同码不同场景），硬规则容易误杀
- 2. 调策略不改代码（改 Prompt = 调阈值）
- 3. 性能用别的手段补（缓存、聚合、异步）

可插拔 Domain 的价值

换团队 = 换配置不动引擎：

- 机票：DOMAIN=flight + domains/flight/
- 酒店：DOMAIN=hotel + domains/hotel/
- 火车票：DOMAIN=train + domains/train/

引擎代码（router/triage/llm/pipeline）对所有团队通用。

路由引擎设计

四步判定：

- 1. pathname 归一化（去 host/query）
- 2. 归属判别优先（非自有前缀 → non-core）
- 3. 有序匹配（特例排在宽松规则前）
- 4. 未命中兜底（fallbackkey 保守上报）

火车票示例：

- 自有前缀：/esapp/
- 分类：payment / order / refund-change / query / user-auth / order-detail / display / non-core
- 兜底：未匹配到规则 → order（P1 保守上报）

2.6 面试高频问题

Q1: 你具体做了什么？

诚实对齐：

- 我做了火车票 Domain 的路由表和 Prompt 配置
- 比如 `/railway/train/transferLeftTicket` 这个接口，我把它路由到 `query` 分类，定义为下单主流程入口，参数错误判 P1

Q2: 工程理念怎么落地的？

讲具体例子：

1. **业务正常拒绝识别**：我去源码里找 `errorCode` 对应的业务场景，把"无票/已退改"写成 Prompt 里的 `business_reject` 清单
2. **兜底策略**：有一次模型解析 JSON 失败了，我加了个 `fallback` 直接保守上报 P2，保证不漏告警
3. **评测纪律**：用漏报率 FN 作为头号指标，宁愿多报噪音也不能漏真故障

Q3: 你觉得这个项目难在哪？

三个难点：

1. **业务知识沉淀**：哪些是业务正常拒绝、哪些是真故障，这个边界要产品+研发一起定义
2. **宁可误报不可误杀**：宁愿多报噪音也不能漏真故障，所以调 Prompt 要保守
3. **接口清单要准**：漏掉的接口可能漏报，所以后来加了 `drift-check` 盯清单漂移

Q4: 为什么余票查询异常判 P1？

回答要点：

- 火车票主流程链：查询(余票) → 填单/校验 → 下单占座 → 支付 → 出票 → 退改
- 余票查询是下单主流程入口，挂了等于阻塞下单
- 不能因为名义上是「查询」就降级成 P2/P3

Q5: 接口清单过期会不会漏报？

诚实回答：

- **不会漏报**：自有前缀内未命中 → `fallbackKey` 保守上报；非自有 → `non-core` 兜底
- 清单过期只降低 `severity` 精度，不会造成漏报
- 后续加了 `drift-check.sh` 纯 `grep` 盯漂移，可挂 CI

Q6: LLM 方案 vs 纯规则方案？

维度	LLM 方案	纯规则方案
优势	能吃语义、策略迭代快、适合灰色地带	延迟低、可硬门禁、成本低
代价	延迟与成本、临界抖动	脆弱、误杀风险高

为何选 LLM：噪音形态多、业务语义重，规则预过滤误杀成本高；先用 LLM 保证召回与可解释 reason。

Q7: 当前落地到哪一步?

诚实现状:

-  通用引擎 (HTTP / Kafka)
-  火车票 Domain (business_reject 已回源核验)
-  机酒 url 清单已摘取 (Domain 待 onboard)
-  真实派发通道 (接 IM / 电话 / 工单, 当前为日志占位)
-  频次统计、告警聚合、结果缓存

面试建议: 主动说清「判定链路已通、派发与缓存是下一阶段」, 体现演进意识而不夸大上线范围。

技术亮点提炼

亮点 1: 双端性能监控架构

问题: H5 和星河小程序技术栈不同, 监控 API 不一样。

方案:

```
1 // Adapter 模式抽象统一上报
2 class PerfAdapter {
3   static create(env) {
4     return env === 'h5' ? new H5Adapter() : new StarAdapter();
5   }
6 }
7
8 class H5Adapter {
9   mark(label) { performance.mark(label); }
10  measure(from, to) { performance.measure(from, to); }
11 }
12
13 class StarAdapter {
14   mark(label) { wx.getPerformance().mark(label); }
15   measure(from, to) { wx.getPerformance().measure(from, to); }
16 }
```

价值: 一套监控代码, 双端复用。

亮点 2: 缓存命中直渲状态机

问题: 列表页切日期, 缓存命中时仍触发网络请求, 导致白屏。

方案:

```
1 // 状态机: cacheHit → skipFetch → directRender
2 async function loadList(date) {
3   const cached = getCache(date);
4   if (cached) {
5     mark('cacheHit');
6     renderList(cached); // 直接渲染
7     mark('directRender');
8     return;
```

```

9     }
10
11     mark('fetchStart');
12     const data = await fetchList(date);
13     mark('fetchEnd');
14     renderList(data);
15 }

```

价值：避免无效网络请求，LCP ↓240ms。

亮点 3：可插拔 Domain 架构

问题：机酒火三个品类接口特征不同，硬编码规则难维护。

方案：

```

1 换团队 = 换 DOMAIN + 加配置文件夹
2 DOMAIN=train  → domains/train/
3 DOMAIN=flight → domains/flight/
4 DOMAIN=hotel  → domains/hotel/

```

价值：引擎代码对所有团队通用，扩展成本 ≈ 0。

亮点 4：LLM 工程化兜底策略

问题：模型超时、网关报错、JSON 解析失败时系统会怎样？

方案：

```

1  async function triage(alarm) {
2      try {
3          const result = await llm.call(prompt);
4          return parseJSON(result);
5      } catch (err) {
6          // 统一兜底：永不吞告警
7          return {
8              should_report: true,
9              severity: 'P2',
10             reason: 'LLM fallback: ' + err.message
11         };
12     }
13 }

```

价值：降噪能力降级时，可用性优先于降噪率。

亮点 5：评测驱动的 Prompt 调优

问题：如何保证降噪不误杀真故障？

方案：

- **漏报率 FN 作为头号指标**（先压到 ≈ 0）
- **数据集冻结：**一轮调优期间不动 dataset
- **train 调 / holdout 验：**holdout 不涨或下跌 = 过拟合

- **正负样本平衡**：防止「一律不报」刷高分

价值：工程化保证降噪质量。

常见面试问题清单

项目一：性能监控

基础理解

1. 请介绍一下页面性能监控项目，解决什么问题？
2. 为什么要做 Skill？和 Lighthouse 有什么区别？
3. PerfMonitor 监控体系具体覆盖哪些指标？

技术细节

4. H5 和星河双端如何统一上报？
5. 列表页 LCP 优化的具体步骤？
6. 为什么要剥离 privacyPopList？
7. 缓存命中直渲的状态机设计？
8. 星河底部菜单卡顿的排查过程？

工程化

9. 如何验证优化效果（LCP ↓17%）？
10. Skill 三阶段模型（加载/交互/监听）如何适配多技术栈？
11. 性能优化 P0/P1/P2 怎么定优先级？

项目二：AI 监控降噪

基础理解

12. 请介绍一下 AI 监控降噪项目，解决什么问题？
13. 为什么每条告警都走 LLM，不做规则预过滤？
14. 「宁可误报不可误杀」在工程上如何落地？

架构设计

15. 系统整体架构是怎样的？引擎和业务怎么解耦？
16. 一条告警从进入到派发经过哪些步骤？
17. 路由引擎怎么设计？ownedPrefixes 和 fallback 起什么作用？
18. 可插拔 Domain 的价值是什么？

技术细节

19. Prompt 是怎么组织的？调判定松紧改哪里？
20. severity（P0-P3）怎么定？为什么余票查询异常判 P1？
21. business_reject（业务正常拒绝）如何识别？

22. LLM 网关故障时系统会怎样？

工程化

- 23. 两个 Skill (discover / onboard) 分别做什么？
- 24. 接口清单过期会不会漏报？drift-check 解决什么问题？
- 25. 评测闭环怎么设计？为什么不看笼统准确率？
- 26. 当前落地到哪一步？后续还有哪些待建设？

对比与综合

- 27. 降噪和故障根因定位有什么区别？
- 28. LLM 方案 vs 纯规则方案的 tradeoff？
- 29. 两个项目的共性和差异？
- 30. 这些数字 (LCP ↓17%) 是你独立做的吗？

避坑指南

✗ 不要这样说

坑1：过度美化参与度

- ✗ "Skill 是我从 0 到 1 写的"
- ✓ "Skill 框架是团队搭的，我参与了监控体系搭建和性能优化落地"

坑2：夸大上线范围

- ✗ "AI 监控降噪已全量上线，降噪率提升 80%"
- ✓ "判定链路已通、火车票 Domain 已回源核验，派发通道与缓存是下一阶段"

坑3：对技术细节含糊其辞

- ✗ "就是用 AI 做了个监控"
- ✓ "基于 LLM 的告警判定系统，每条走一次模型，输出 should_report / severity / reason"

坑4：无法回答「你具体做了什么」

- ✗ "就是配置了一下 Prompt"
- ✓ "我做了火车票 Domain 的路由表、severity 基线、business_reject 清单，并回源码核验"

坑5：把估算当真测

- ✗ "Skill 已经精确测出所有函数耗时"
- ✓ "Skill 用类型估算表给出优先级，P0 改动后必须用 DevTools / 真机校准"

✅ 加分表述

加分1：主动说边界

- "PerfMonitor 负责线上验收, Skill 负责缩小嫌疑"
- "降噪解决注意力分配, 不替代故障根因定位"

加分2：展示演进意识

- "阶段一内嵌验证判定效果, 阶段二中心化配置推广"
- "当前重点是压 FN 到 ≈ 0 , 下阶段再降 FP"

加分3：体现工程思维

- "引擎领域无关, 业务知识在可插拔 Domain"
- "每条都走 LLM 保证质量, 性能靠缓存/聚合补"

加分4：量化结果 + 方法论

- "LCP 1400 $\rightarrow$ 1160ms ($\downarrow$ 17%), 通过 Omega 线上统计"
- "漏报率 FN 作为头号指标, train 调 / holdout 验"

快速复习 Checklist

面试前 30 分钟

项目一：性能监控

- ☐ 能说出 LCP $\downarrow$ 17%、卡顿 $\downarrow$ 27% 的具体优化动作
- ☐ 能画出 PerfMonitor 架构 (三段式 API + 双端 Adapter)
- ☐ 能解释为什么剥离 privacyPopList
- ☐ 能说出 Skill 三阶段模型 (加载/交互/监听)

项目二：AI 监控降噪

- ☐ 能说出系统分层 (入口/pipeline/router/triage/llm/domain)
- ☐ 能解释为什么每条都走 LLM (不做规则预过滤)
- ☐ 能说出火车票主流程链 (查 $\rightarrow$ 填 $\rightarrow$ 下 $\rightarrow$ 付 $\rightarrow$ 票 $\rightarrow$ 退改)
- ☐ 能解释余票查询异常为什么判 P1
- ☐ 能说出 FN 作为头号指标的评测纪律

技术亮点

- ☐ 双端性能监控 Adapter 模式
- ☐ 缓存命中直渲状态机
- ☐ 可插拔 Domain 架构
- ☐ LLM 兜底策略 (永不吞告警)
- ☐ 评测驱动 Prompt 调优

避坑

- ☐ 不夸大参与度 ("团队搭的 + 我落地的")
- ☐ 不夸大上线范围 ("判定链路已通 + 派发是下一阶段")
- ☐ 能诚实说「这块我不太清楚，但可以讲我负责的部分」

附录：关键数字速查

指标	数值	来源
列表页 LCP 优化	1400ms → 1160ms (↓17%)	Omega 线上统计
星河底部菜单卡顿	改善约 27%	用户行为埋点对比
实习时长	7 个月 (2026.01–2026.07)	实习记录
监控覆盖	跨页/LCP/FCP/接口维度	perf-monitor.js
火车票主流程	查→填→下→付→票→退改	业务文档
评测头号指标	漏报率 FN ≈ 0	eval 闭环

文档使用说明

面试前一天

- 通读全文，重点记忆「技术亮点提炼」和「常见面试问题清单」
- 用「快速复习 Checklist」自查，确保核心问题都能回答

面试前 1 小时

- 只看「快速复习 Checklist」和「关键数字速查」
- 在白纸上默画两个项目的架构图

面试中

- 优先讲「问题背景 → 技术方案 → 量化结果」
- 遇到不会的追问，诚实说「这块我不太清楚，但可以讲我负责的部分」
- 主动展示「边界意识」和「演进思维」

祝面试顺利！🚀